

Web Application Vulnerability Scanner Report

1. Introduction

Web applications are widely used for online services such as banking, shopping, and communication. However, many web applications are vulnerable to attacks such as Cross-Site Scripting (XSS), SQL Injection (SQLi), and Cross-Site Request Forgery (CSRF). These vulnerabilities can allow attackers to steal data, manipulate databases, or perform unauthorized actions. The objective of this project is to build a Web Application Vulnerability Scanner that can detect common web vulnerabilities and help improve web security.

2. Abstract

This project presents a Web Application Vulnerability Scanner developed using Python and Flask. The scanner crawls web pages, identifies input fields and URLs, and tests them for common vulnerabilities such as XSS, SQL Injection, and CSRF. The system injects test payloads into input fields and analyzes the server response to detect vulnerabilities. The results are displayed in a web interface along with vulnerability details, severity level, and evidence. This project helps in understanding web security testing and vulnerability assessment techniques.

3. Tools Used

Python, Flask, requests library, BeautifulSoup, OWASP Top 10 checklist, Regular Expressions (Regex), Web Browser.

4. Steps Involved in Building the Project

1. Install Python and required libraries such as requests, BeautifulSoup, and Flask. 2. Create a crawler to collect URLs and input fields from the target website. 3. Develop payloads for XSS, SQL Injection, and CSRF testing. 4. Send payloads to the target web application using HTTP requests. 5. Analyze the response using pattern matching and regex to detect vulnerabilities. 6. Assign severity levels (Low, Medium, High) based on vulnerability type. 7. Store scan results and evidence in a log file. 8. Create a Flask web interface to start scans and display results. 9. Generate a report of detected vulnerabilities.

5. Conclusion

The Web Application Vulnerability Scanner successfully detects common web vulnerabilities such as XSS, SQL Injection, and CSRF. The project helps in understanding how web vulnerabilities are identified and how security testing tools work. The Flask-based interface makes the tool easy to use and understand. This project can be further improved by adding more vulnerability checks, automation features, and report generation options.